

Beyond Structure-Driven Tuning: Cost-Aligned Graph Optimization for Approximate Nearest Neighbor Search

Zhiwei Zhang

School of Data Science
Fudan University, Shanghai, China
zwzhang25@m.fudan.edu.cn

Weiguo Zheng*

School of Data Science
Fudan University, Shanghai, China
zhengweiguo@fudan.edu.cn

Abstract

Graph-based approximate nearest neighbor search (ANNS) is widely used in vector databases and retrieval systems. Most existing methods typically rely on structure-driven index construction and tuning, building graph topologies that approximate idealized geometric archetypes to promote navigability. However, geometric proximity is often an indirect proxy for search efficiency: static graph structures may fail to reflect actual traversal dynamics, leading to redundant distance evaluations such as low-yield node expansions and unnecessary local detours. These inefficiencies can significantly hinder high-recall performance. To address this limitation, we introduce PIGR (Post-hoc Iterative Graph Refinement), a trace-driven framework that explicitly aligns efficiency with observed search behavior. Instead of relying on coarse-grained construction parameter tuning, PIGR leverages self-queries to analyze search traces and identify inefficient traversal behaviors. It further performs iterative index optimization via trace-guided, budgeted prune-and-add edge edits without modifying the deployed search procedure. As a post-construction plugin, PIGR delivers $1.3\times$ – $2.5\times$ speedups at near-exact recall across diverse datasets and index families, consistently outperforming the best build-time tuned baselines. All code is available at <https://github.com/codenoobzzw/PIGR>.

CCS Concepts

• Information systems → Information retrieval query processing.

Keywords

Approximate nearest neighbor search, post-hoc iterative index refinement, trace-driven optimization

ACM Reference Format:

Zhiwei Zhang and Weiguo Zheng. 2026. Beyond Structure-Driven Tuning: Cost-Aligned Graph Optimization for Approximate Nearest Neighbor Search. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea*. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3770855.3817671>

*Weiguo Zheng is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3817671>

1 Introduction

Approximate nearest neighbor search (ANNS) is becoming increasingly popular in various application domains, such as information retrieval [28, 42], pattern recognition [4, 19], recommendation systems [26, 29], and retrieval augmented generation (RAG) [11, 25]. Given a query vector q , ANNS aims to return the k database vectors that are approximately nearest to q under a distance metric. Practical systems therefore build an *index* and run a fixed *search procedure* that probes only a small fraction of points while meeting a target recall. Among major ANNS paradigms such as quantization-based [17], hashing-based [15, 22], tree-based [27, 30], and graph-based indexes [7, 24, 31, 33, 36], graph-based indexes are particularly competitive due to their strong accuracy–efficiency trade-offs and favorable memory-access behavior [24, 31, 36].

Graph indexes represent data points as vertices and connect each vertex to a small set of selected neighbors. At query time, most graph indexes follow a similar traversal template: starting from one (or a few) entry points, the procedure repeatedly expands candidates along edges using greedy routing or beam-style best-first exploration, guided by distance evaluations between q and visited candidates [24, 31, 36]. This shared template makes graph search simple and effective, and it also clarifies where the cost comes from: under a fixed traversal discipline, the main cost is the number of distance evaluations triggered before the candidate set converges to approximately nearest neighbors [3, 20].

Mainstream graph builders typically start from an intended *graph archetype* and then design neighbor selection and pruning rules so that the resulting edge set approximates that archetype. For instance, NSG promotes RNG-like [16, 32] sparsification (a proximity-graph rule that removes “dominated” edges blocked by closer intermediates) to enhance navigability [7], HCNNG uses clustering and connectivity structure (MST graph [14]) to guide a hierarchical construction [33], and HNSW leverages a multi-layer small-world design [2, 24]. Such structure-driven construction has indeed delivered strong ANNS performance and enabled widely used systems [35, 37]. However, systematic comparisons also indicate that the relative performance of different graph families varies substantially across datasets and data distributions [36]. This suggests that there is no single, universally optimal topology that one should always “move toward.” More fundamentally, *a graph being closer to a particular archetype does not directly imply higher QPS (Queries Per Second): throughput is determined by the realized traversal, i.e., how many distance evaluations are performed and how many of them lead to effective progress in the candidate set*. Therefore, rather than optimizing the graph index for a particular topology, we focus on how to systematically reduce distance evaluations without modifying the search procedure while maintaining high recall.

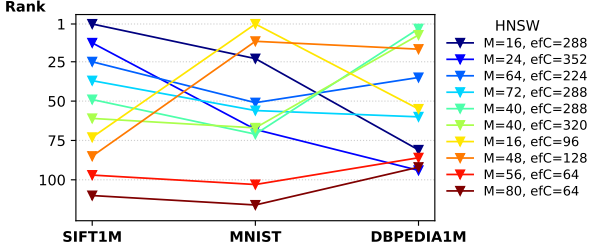


Figure 1: Best configurations differ across datasets (QPS–Recall@10), more configurations see Figure 9.

1.1 Motivation

Prevailing practice: tuning and rebuilding as the primary route to higher QPS. To increase query throughput at a given recall target, it is common to rely on iterative tuning of construction-time hyperparameters followed by repeated index rebuilding. In most graph-based index builders, these hyperparameters govern the global characteristics of the edge set in a coarse-grained fashion, such as the maximum out-degree and the size of the candidate set used during neighbor selection. The types of these hyperparameters and what they control differ across index families.

Nevertheless, identifying an optimal hyperparameter configuration in advance is often difficult and highly data dependent. Our empirical sweeps on HNSW confirm this observation: even within a single graph family, the parameter settings that deliver the best recall–QPS trade-off differ substantially across datasets as shown in Figure 1. Advancing from a minimally acceptable configuration to a high-quality one typically requires multiple rebuild iterations. As a result, structure-driven graph tuning tends to incur a long time-to-quality and exhibits strong sensitivity to the underlying data distribution. In practice, both the optimal parameter choices—and, at times, even the relative performance ranking among graph families—can vary substantially across datasets.

Interpretation: a black-box search over a builder-induced, structure-biased graph space. Importantly, the tuning loop is usually driven by coarse, aggregate feedback: one modifies a hyperparameter, rebuilds the graph, evaluates QPS–recall on a validation set, and then decides the next modification, or simply sweeps a grid of candidate settings. From a structural perspective, these hyperparameters move the index within a large graph space induced by the builder—a space that encodes the builder’s structural preferences. Since the mapping from hyperparameters to high-recall efficiency is rarely transparent, practitioners often cannot determine in advance whether the parameter should be increased or decreased to reduce query-time cost. Thus, sweep-style trial-and-rebuild becomes the easy and direct mechanism for improvement. This prevalence also underscores a key fact: modifying the edge set indeed changes the realized search behavior, but what remains missing is a cost-aligned principle that prescribes how the edge set should be changed when high-recall efficiency is unsatisfactory.

Insight: search traces provide fine-grained and cost-aligned evidence. We observe that the traversal itself induces *search traces* that are naturally aligned with query-time cost. In particular, traces often reveal two recurring inefficiency patterns that become especially salient in the high-recall regime. (i) *Low-yield expansions and redundant edges.* Certain edges are activated frequently during traversal, yet the associated evaluations rarely lead to a meaningful

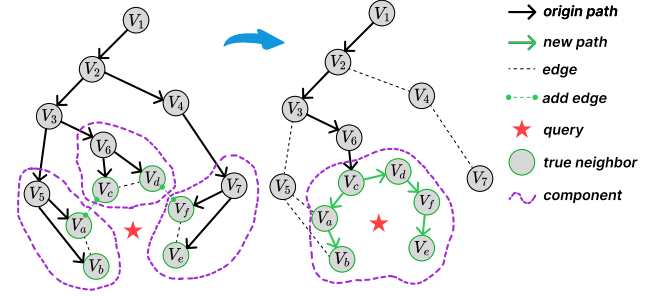


Figure 2: Detours under neighborhood fragmentation (left) and evaluation reduction via bridging (right).

tightening of the candidate set (e.g., they seldom improve the best-first/beam frontier). As a result, a non-trivial fraction of the search budget is consumed by expansions with persistently low marginal utility. (ii) *Detours induced by local fractures.* As illustrated in Figure 2, when the query-adjacent neighborhood is weakly connected or locally fragmented in the graph, the traversal may fall back to earlier candidates (e.g., v_2 and v_3) and expand outward to reach other relevant regions, resulting in repeated enter–leave–reenter detours. Under high-recall requirements, the backtracking caused by local neighborhood fractures tends to occur more frequently, amplifying redundant expansions and distance evaluations [21].

Together, these observations suggest that aggregate QPS–recall is too coarse to explain where distance evaluations are incurred and why the traversal exhibits redundant work. What is missing is cost-aligned evidence that connects observed trace behaviors back to the edge set, so that we can probabilistically prescribe how to change the edge set when high-recall efficiency is unsatisfactory.

1.2 Our Approach and Contributions

We introduce a trace-feedback-driven refinement framework **PIGR** (Post-hoc Iterative Graph Refinement) that enhances an existing graph index without modifying the deployed search procedure. Starting from an index constructed by an initial graph builder, we execute a set of self-queries and collect the corresponding search traces. These traces offer cost-aligned insights into the actual traversal behavior, revealing where distance evaluations are concentrated and identifying repeated low-yield expansions.

Guided by this evidence, we derive fine-grained signals that map trace behaviors back to specific edges in the graph, targeting two recurring sources of inefficiency: persistently low-yield expansions and detours/backtracking caused by insufficient connectivity. Leveraging these signals, we apply budgeted modifications to the edge set. Specifically, the refinement applies two complementary operations: (i) pruning edges that consistently trigger distance evaluations with little progress under the deployed traversal policy, and (ii) inserting high-impact connections that mitigate systematic detours and accelerate search.

By iteratively collecting traces and performing budgeted edge updates over a small number of rounds, the refinement process improves high-recall throughput by reducing the number of distance evaluations required for search, without constraining the optimization to any predefined topological pattern.

In summary, we make the following contributions.

- **A new trace-guided viewpoint for refining graph ANNS.** We reduce realized distance-evaluation cost via search-trace analysis

Table 1: Summary of notations

Notation	Description
S	Base dataset in Euclidean space $\mathcal{E}^d$
q	A query point in the query workload
$G = (V, E)$	Directed graph index with vertex V and edge E
$e = (u, v)$	A directed edge from u to v
$N(v)$	Out-neighbor set of vertex v in G
k	Number of nearest neighbors to return
$K(q)$	Ground-truth top- k neighbors of q
R	Retrieved result set
τ	Target recall level
H_R	Bounded best list
ef	Beam size in greedy beam search
$\text{dis}(\cdot, \cdot)$	Euclidean distance
$\text{De}(G, q)$	Number of distance evaluations for query q on G
B	Pruning budget

to directly improve search performance at high recall, rather than adjusting the graph toward a pre-specified topology.

- **A trace-driven refinement framework with budgeted edge editing.** We introduce a framework that leverages search traces as feedback to diagnose traversal inefficiencies and performs budgeted edge edits to reduce redundant distance evaluations without changing the query procedure.
- **An integration-light, plug-in upgrade technique.** Our refinement can be inserted after construction and applied to different index families and parameters, reducing the need for parameter sweeps and repeated rebuilds.
- **Consistent improvements in the high-recall regime.** Across multiple datasets and graph families, our approach consistently improves performance at high recall, and these improvements remain robust under different parameter settings and query workloads.

2 Preliminaries

2.1 Problem and Definitions

In this paper, we consider a directed proximity graph index $G = (V, E)$ built over a dataset $S \subset \mathcal{E}^d$, where each vertex corresponds to a data point and each directed edge $e = (u, v)$ indicates a candidate neighbor relation; $N(v)$ denotes the out-neighbors of v . Queries are drawn from a workload Q , and for a query $q \in Q$ we denote its exact top- k neighbors in S by $K(q)$ and the returned set by R . Accuracy is measured by Recall@ k , defined as $\text{Rec}_k(G, q) = |K(q) \cap R|/k$ [4]. We measure efficiency by throughput in queries per second (QPS), and additionally use the number of distance evaluations $\text{De}(G, q)$ as a hardware-agnostic proxy for query cost. The frequently-used notations are summarized in Table 1.

Definition 2.1 (Graph-Based ANNS). Given $S \subset \mathcal{E}^d$, graph-based ANNS constructs a directed graph index $G = (V, E)$ over S . For a query q , it performs search on G and returns a size- k set $R \subseteq S$ that approximates the exact top- k neighbors $K(q)$.

2.2 Base Algorithms

Baseline indexes. Representative graph-based ANNS indexes are used as baselines, including NSG [7], HCNNG [33], and HNSW [24]. These indexes differ in the construction heuristics and the resulting graph structures, and are widely adopted in practice [36]. NSG

targets a sparse navigable graph with an explicit entry point [7]; HCNNG introduces hierarchical clustering to organize local backbones [33]; HNSW uses a multi-layer structure with greedy descent followed by beam-style search at the base layer [24].

Shared query kernel: greedy beam search. Although NSG, HCNNG, and HNSW differ in construction and use index-specific components such as entry-point selection, hierarchical descent, or guided neighbor filtering, their search procedure shares a common expansion rule. We abstract this shared rule as *greedy beam search*: given an index G and a query q , the procedure maintains a candidate queue ordered by similarity (e.g., Euclidean distance $\text{dis}(\cdot, q)$), and a bounded best list H_R of the currently closest candidates. It repeatedly selects the closest unexpanded candidate, expands its neighbors, and inserts newly discovered vertices if they can tighten the current beam threshold induced by the best list; the procedure terminates when no remaining candidate can improve the best list under the beam constraint. Finally, the k closest vertices in the best list are returned as R [23]. Core pseudocode is given in Appendix A.

3 Formulation and Refinement Principles

In this section, we formalize a fixed-recall, cost-aligned objective for graph-based retrieval, and derive refinement principles that indicate how edge edits can reduce distance evaluations.

3.1 Optimization Objective

Previous work has observed that query latency in graph-based ANNS is strongly correlated with the number of distance evaluations [6, 10, 20]. Given a query workload Q , we define

$$\min_{G \in \mathcal{G}(S)} \mathbb{E}_{q \sim Q} [\text{De}(G, q)] \quad \text{s.t.} \quad \text{Rec}_k(G; Q) \geq \tau \quad (1)$$

where $\mathbb{E}_{q \sim Q}[\cdot]$ denotes $\frac{1}{|Q|} \sum_{q \in Q} (\cdot)$ and $\mathcal{G}(S)$ denotes the space of graph topologies over S .

Accordingly, PIGR is designed to improve efficiency by reducing the distance evaluations required to achieve the target recall. Throughout this paper, we instantiate $\text{De}(G, q)$ using the (index-specific) greedy beam search in our experiments, but the objective itself can be defined for any search discipline by interpreting $\text{De}(G, q)$ as the distance evaluation count under the corresponding discipline. At a high level, PIGR uses Equation (1) as a cost-aligned guiding objective and refines the graph through an iterative refinement loop: it collects search traces on the graph index, uses them as feedback to edit the edge set, and repeats for a small number of rounds (detailed in Section 4). We do not claim that PIGR solves Eq. (1) to global optimality, nor do we provide convergence or approximation guarantees. Rather, Eq. (1) serves as a fixed-recall, cost-aligned design target. PIGR instantiates this target through trace-derived edge-editing rules under a fixed search discipline.

3.2 Edge-Level Efficiency and Pruning

Efficiency from search to edge. In expansion-based graph search algorithms, such as greedy beam search, each query triggers a sequence of distance evaluations on neighbors of expanded nodes. We categorize these evaluations by whether they are accepted into the bounded best list at the time of evaluation:

- (1) **Adopted:** distance evaluations that insert the evaluated neighbor into H_R (tightening the current H_R threshold).

(2) **Discarded**: distance evaluations that do not change H_R .

Let $\text{adopt}(s)$ and $\text{discard}(s)$ denote the numbers of adopted and discarded evaluations of a single search instance s , respectively. We define its *efficient evaluation rate* as

$$\text{EER}(s) = \frac{\text{adopt}(s)}{\text{adopt}(s) + \text{discard}(s)}. \quad (2)$$

A higher $\text{EER}(s)$ means that a larger fraction of distance evaluations are adopted into H_R , i.e., they more often tighten the H_R threshold, indicating more effective progress per evaluation.

By aggregating statistics over many searches on a fixed graph G , we obtain edge usage information. Importantly, we record an edge event only when the endpoint distance is actually evaluated; edges filtered before evaluation do not contribute to $\text{De}(G, q)$ and are excluded from the EER accounting. For an edge $e \in E$, let $\text{adopt}(e)$ and $\text{discard}(e)$ be the total numbers of adopted and discarded evaluations associated with e in all search traces. The *edge-level efficient rate* is then determined:

$$\text{EER}(e) = \frac{\text{adopt}(e)}{\text{adopt}(e) + \text{discard}(e)}. \quad (3)$$

Interpreting these counts as empirical trace statistics, $\text{EER}(e)$ can be viewed as the adoption frequency of evaluations triggered by e .

From an edge-centric perspective, a high $\text{EER}(e)$ means that evaluations through e frequently tighten the H_R threshold, while a low $\text{EER}(e)$ indicates that e often incurs distance evaluations without immediate benefit. Such low-EER edges are natural candidates for pruning when the goal is to reduce distance evaluations under a fixed-recall constraint. We provide a conservative analysis under a fixed traversal and a recall-preserving pruning step in Appendix B.

3.3 Detours and Bridging Edges

Detours from fragmented neighborhoods. In graph-based ANNS, search traversal is often expected to move toward nearest neighbors by making monotonic improvements [7]. However, the near-neighbor set around $K(q)$ may be split into several neighborhood components that are well connected internally but weakly connected to each other. Once the traversal enters one component, it may expand deeply within it and then fall back to older beam candidates to reach another relevant component, causing repeated “enter–leave–reenter” detours and many redundant distance evaluations. The left panel of Figure 2 shows a fragmented neighborhood: $V_a, \dots, V_f$ form three weakly connected components, so the traversal must fall back to earlier beam candidates (e.g., V_2 and V_3) to reach others, often requiring a larger beam to maintain recall.

Bridging fragmented neighborhoods. A natural refinement is to add a small number of *bridging edges* that connect such weakly connected components, creating short cross-component paths for the deployed traversal. In PIGR, bridge candidates are proposed from query traces: if returned neighbors are mutually reachable within each trace-induced component but only weakly connected across components, it indicates missing bridges between the corresponding components. In the right panel of Figure 2, bridges from V_c to V_a and from V_d to V_f provide direct cross-component shortcuts, reducing reliance on earlier beam candidates (e.g., V_2 and V_3) and thereby lowering backtracking and redundant distance evaluations; see Appendix C for a detailed explanation of how reducing detours translates into fewer distance evaluations. Because added bridges

Algorithm 1 EER-Guided Pruning($G, S, \rho, ef_{\text{prune}}$)

Require: Index $G = (V, E)$, base S , pruning ratio ρ , beam ef_{prune}
Ensure: Pruned graph G'

```

1:  $\text{adopt}[e] \leftarrow 0, \text{discard}[e] \leftarrow 0 \ \forall e \in E$ 
2: for  $q \in S$  do
3:   for search on  $G$  with beam  $ef_{\text{prune}}$  do
4:     if adopted into  $H_R$  then  $\text{adopt}[e] \leftarrow \text{adopt}[e] + 1$ 
5:     else  $\text{discard}[e] \leftarrow \text{discard}[e] + 1$ 
6:    $C \leftarrow \{e \in E : \text{adopt}[e] + \text{discard}[e] > 0\}$ 
7:    $\text{score}(e) \leftarrow \begin{cases} +\infty, & \text{adopt}[e] = 0 \\ \text{discard}[e]/\text{adopt}[e], & \text{otherwise} \end{cases} \ \forall e \in C$ 
8:    $B \leftarrow \lfloor \rho|E| \rfloor, \mathcal{P} \leftarrow \text{TOP}(C, B)$  by decreasing  $\text{score}$ 
9: return  $G' = (V, E - \mathcal{P})$ 
```

can also be judged by the same EER statistics, effective bridges tend to survive later pruning while ineffective ones are removed, preventing uncontrolled densification.

3.4 Generalization Beyond Self-queries

A graph index is built on a base set S , while deployment queries may come from a separate set outside S . PIGR refines the index by running self-queries on points in S and updating the graph structure G . Consequently, after refinement, all queries—whether $q \in S$ (a self-query) or an external query point $q \notin S$ —are served by the same refined graph under the same search procedure. This can benefit deployment queries because the graph topology governs the neighbor evaluations and detours incurred by the traversal under the fixed search procedure. Therefore, refining G to provide more direct routes and fewer structural detours can systematically reduce redundant expansions, thereby reducing distance evaluations, for both self-queries and deployment queries.

4 Cost-Aligned Refinement

Figure 3 provides the overview of the PIGR pipeline. Starting from a built index G , PIGR runs self-queries on G and analyzes search traces to refine the index. Refinement proceeds in iterations, each performing a single edge-editing operation—either prune or add—and following a fixed alternation that always starts with prune and then add (prune→add→prune→...). We refer to one consecutive prune + add pair as a refinement *round*. The output of each iteration serves as the input to the next.

4.1 EER-Guided Edge Pruning

In each refinement round, PIGR starts with pruning. The goal is to reduce distance evaluations under the deployed search by removing statistically low-yield edges according to trace-derived EER feedback. Algorithm 1 summarizes the procedure.

We collect pruning traces by running the search on the current graph G using base points as self-queries. Specifically, for each $q \in S$ we execute a search with size ef_{prune} and record the evaluated directed edges. Then we compute $\text{EER}(e)$ for each edge from these traces following the definition in Section 3.2, and prune statistically low-yield edges accordingly.

Example 4.1. The PRUNE panel of Figure 3 shows a toy *Recall@2* trace. Each evaluated edge yields a binary outcome: green edges correspond to *adopted* evaluations that insert a candidate into H_R ,

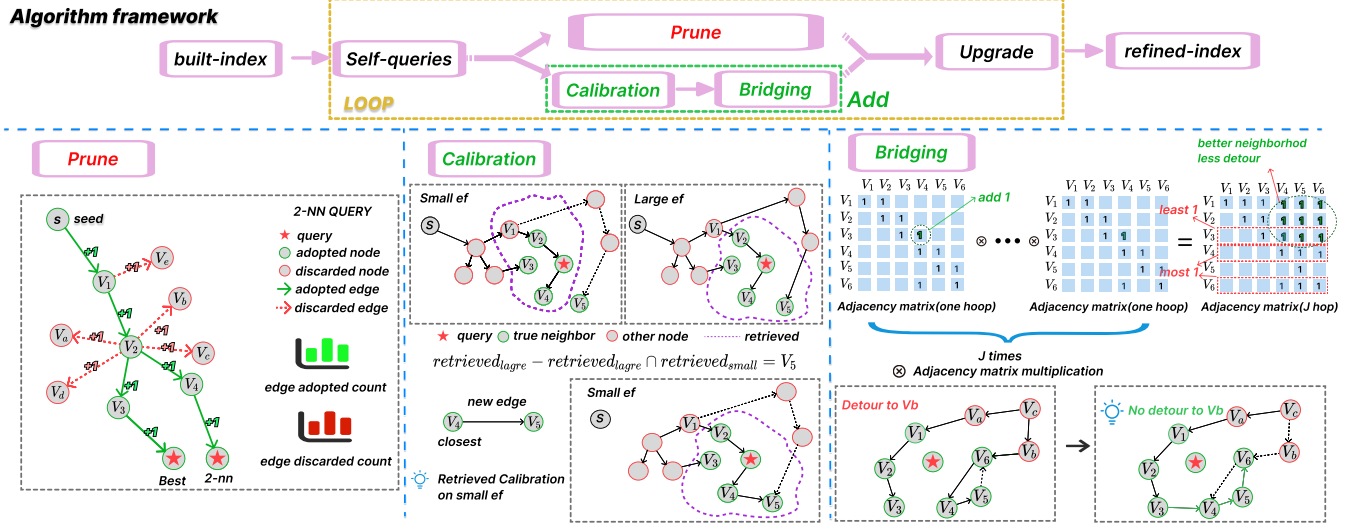


Figure 3: PIGR refinement loop. Self-queries generate traces that alternately drive EER-guided pruning and a two-stage add module (calibration → bridging) to reduce distance evaluations under the deployed search procedure.

Algorithm 2 PIGR-Calibration ($G, S, k, ef_{gc}, ef_{target}$)

Require: Index $G = (V, E)$, base S , neighbor count k , large beam size ef_{gc} , small beam size ef_{target}

Ensure: Updated index $G' = (V, E')$

- 1: **for** each $v \in S$ **do** // v acts as a self-query
- 2: Obtain $HCN(v)$ using greedy beam search with (q, k, ef_{gc})
- 3: Obtain $LCN(v)$ using greedy beam search with (q, k, ef_{target})
- 4: $O(v) \leftarrow HCN(v) \cap LCN(v)$
- 5: **for** each $w \in HCN(v) \setminus LCN(v)$ **do**
- 6: $u^*(w) \leftarrow \arg \min_{u \in O(v)} \text{dis}(u, w)$
- 7: Add $(u^*(w), w)$ to E'
- 8: **return** $G' = (V, E')$

Algorithm 3 PIGR-Bridge-Enhancement (G, S, ef_{add}, k, J, R)

Require: Index $G = (V, E)$, base S , beam size ef_{add} , size of returned neighbors k , hop J , repetitions R

Ensure: Updated index $G' = (V, E')$

- 1: **for** each node $u \in V$ **do**
- 2: Run search for u with size ef_{add} and obtain $R(u)$ of size K
- 3: Build trace-induced adjacency matrix A_0 over $R(u)$
- 4: Compute J -hop reachability $A^{(J)}$ from A_0
- 5: Identify x_{i*} with weakest J -hop connectivity in $A^{(J)}$
- 6: Select x_{j*} maximizing the J -hop connectivity gain for x_{i*}
- 7: **if** $(x_{i*}, x_{j*}) \notin E'$ **then** add (x_{i*}, x_{j*}) to E'
- 8: **if** $(x_{j*}, x_{i*}) \notin E'$ **then** add (x_{j*}, x_{i*}) to E'
- 9: **return** $G' = (V, E')$

whereas red edges correspond to evaluations that are discarded by H_R . Aggregating these events over many self-queries yields per-edge statistics and thus $EER(e)$.

We handle edges based on trace support. Edges with $\text{adopt}(e) = 0$ and $\text{discard}(e) > 0$ are deterministically unproductive under the observed traversal and are prioritized for pruning. Edges that are never evaluated in traces (i.e., $\text{adopt}(e) = \text{discard}(e) = 0$) provide no evidence and are excluded from the pruning candidates. For all remaining trace-supported edges, we assign a pruning score $r(e) = \text{discard}(e) / \text{adopt}(e)$ (equivalently, sorting by increasing $EER(e)$) and prune the worst-scoring ones. Given a pruning ratio ρ , we remove at most $B = \lfloor \rho |E| \rfloor$ directed edges per iteration: we prune case-(i) edges first, and if budget remains, we fill it with the largest- $r(e)$ edges among the trace-supported set.

4.2 Two-Stage Edge Addition

The edge-addition module targets detour reduction under the fixed deployed traversal by injecting a small, budgeted set of shortcut edges. As illustrated in Figure 3, we realize this addition step as a two-stage unit; see Algorithms 2 and 3.

Why addition is two-stage. PIGR is trace-driven: both pruning signals and addition candidates are inferred from what the deployed beam search actually visits. Under a small beam size, detours arise both from missed early entrance into nearby relevant components and from fragmented connectivity within the exposed neighborhood. When entrance is difficult, traces can be dominated by late entry and backtracking, which biases the intra-neighborhood connectivity inferred from traces. We therefore add edges as a two-stage unit: *calibration* first places low-cost gateways to ease entrance, and *bridging* then repairs residual intra-neighborhood fragmentation using post-calibration traces.

Stage 1 (Calibration). For each node v as a self-query (treating the node as the query, i.e., $q = v$), we run search with a large beam size ef_{gc} to obtain a top- k reference set $HCN(v)$, and with the target beam size ef_{target} to obtain $LCN(v)$. Nodes in $HCN(v) \setminus LCN(v)$ indicate nearby components that the target beam tends to under-reach. For such node w , we attach it from an anchor in the overlap $O(v) = HCN(v) \cap LCN(v)$: $u^*(w) = \arg \min_{u \in O(v)} \text{dis}(u, w)$, and add $(u^*(w), w)$. These attachment edges act as gateways: once the traversal reaches $O(v)$ under the small beam, it is more likely to

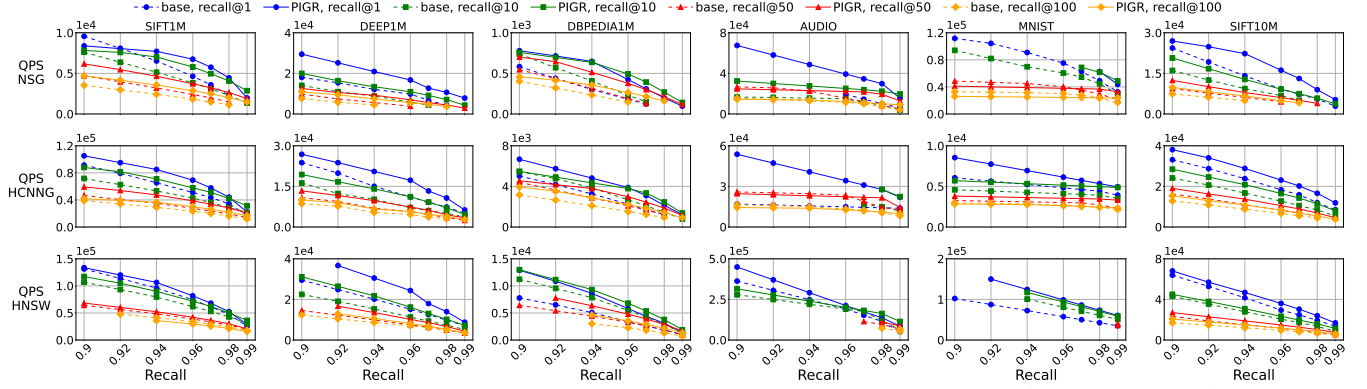


Figure 4: Overall QPS–Recall@ k performance across datasets and index structures ($k \in \{1, 10, 50, 100\}$).

enter the previously under-reached components via a short route (Algorithm 2). For example, in the CALIBRATION panel of Figure 3, treating V_4 as such a missed node w , adding the attachment edge to its nearest anchor (e.g., V_5) allows the target small- ef traversal to reach that under-reached component and reduce entrance detour.

Stage II (Bridging). After calibration, we run self-queries and collect traces on the updated index with beam size ef_{add} . For each node u , we run the search and take the returned top- k candidates as $R(u)$, then build a trace-induced adjacency over $R(u)$ from the local expansions observed along the search. If $R(u)$ splits into weakly connected components under J -hop reachability, bridging inserts a small number of shortcut edges to connect these components by linking the locally weakest-connected node to the partner that maximizes the J -hop connectivity gain (Algorithm 3). As illustrated in the BRIDGING panel of Figure 3, the trace-induced adjacency over $R(u)$ reveals two weakly connected components (e.g., $\{V_1, V_2, V_3\}$ and $\{V_4, V_5, V_6\}$) under J -hop reachability. Adding a shortcut such as $V_3 \rightarrow V_4$ connects the components in this local view, allowing the traversal to move across components without backtracking to V_c , thereby reducing detours.

Beam size selection. All beam sizes used in refinement (e.g., ef_{gc} , ef_{target} , and ef_{prune}) are chosen by a base-only calibration procedure; see Appendix D.

Complexity Analysis. We measure refinement cost under the deployed greedy beam search kernel using the number of distance evaluations, which dominates runtime in graph-based ANNS. Let S denote the base set used as self-queries and let $\mathcal{U}$ denote seed set for bridging. For beam size ef , let $De_{ef}(q)$ be the distance-evaluation count of greedy beam search for query q on the current graph, and define $ef_{\max} = \max\{ef_{prune}, ef_{gc}, ef_{target}, ef_{add}\}$. Let E_{act} be the set of directed edges that appear in traces. In bridging (Algorithm 3), k is the size of the returned neighbor set and J is the hop parameter for reachability; the procedure computes J -hop reachability once per seed on the trace-induced graph and performs at most a constant number of edge insertions.

For one refinement round, define the distance-evaluation totals of the three trace sweeps as $D_{PRUNE} = \sum_{q \in S} De_{ef_{prune}}(q)$, $D_{CAL} = \sum_{v \in S} (De_{ef_{gc}}(v) + De_{ef_{target}}(v))$, and $D_{BRIDGE} = \sum_{u \in \mathcal{U}} De_{ef_{add}}(u)$. Let $D_{ATTACH} = \sum_{v \in S} \sum_{w \in HCN(v) \setminus LCN(v)} |O(v)|$ be the attachment-selection overhead in calibration, which satisfies $D_{ATTACH} \leq |S| \cdot k^2$ (or $\leq |S| \cdot b_{gc} \cdot k$ if we cap the number of attachments per v). Therefore, the time cost of one round is $O((D_{PRUNE} + D_{CAL} + D_{BRIDGE} + D_{ATTACH}))$.

$\log ef_{\max} + |E_{act}| \log |E_{act}| + |\mathcal{U}| J k^3$). The per-round (prune and add) overhead is essentially a few full search sweeps over all base points. Over T refinement rounds, the total cost scales linearly as T times the above; see Appendix E for the detailed derivation.

5 Evaluation

In this section, we evaluate PIGR as a post-hoc refinement framework for improving high-recall efficiency of pre-built graph indexes. For each dataset, we execute the standard query workload to characterize the efficiency–accuracy trade-off (QPS vs. Recall@ k), while keeping the index-specific search procedure unchanged. We generate the Pareto frontier by sweeping only the query-time beam size (ef) for performance comparison [1]. To mitigate system noise, all reported measurements are averaged over five independent runs. We organize the evaluation around five questions:

(Q1) Generality across structures and datasets. Can PIGR improve high-recall search efficiency consistently across structurally different graph families and diverse dataset modalities?

(Q2) Stabilization behavior. How quickly does the iterative refinement stabilize, and how stable is its behavior across iterations?

(Q3) Interaction with build-time tuning. Can PIGR further improve a best-tuned construction, and can it upgrade a typical construction to match or even surpass the best-tuned baseline?

(Q4) Sources of improvement. Are the gains merely due to increasing or decreasing the number of edges?

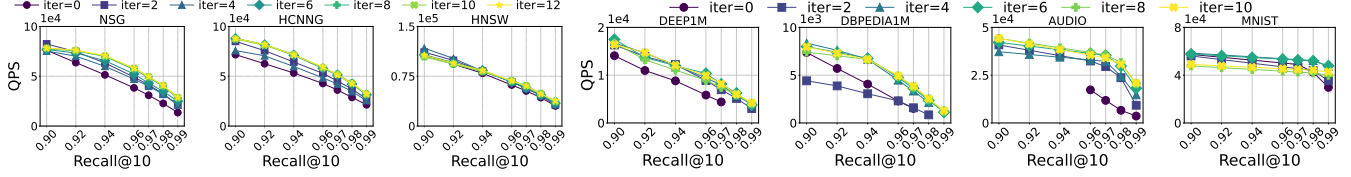
(Q5) Additional graph families and edge-editing baselines. Is PIGR effective on recent graph variants and competitive with other similar edge-editing baselines?

Additional parameter ablations are provided in Appendix K.

5.1 Experimental Setup

Datasets. We evaluate PIGR on six datasets: SIFT1M, DEEP1M, DBPEDIA1M, MNIST, AUDIO and SIFT10M. Their statistics are summarized in Table 10. SIFT1M, SIFT10M and DEEP1M are large-scale visual embedding datasets with medium-dimensional vectors. DBPEDIA1M is a high-dimensional text embedding dataset with 1M Wikipedia articles [5]. MNIST is a visual dataset with a high-dimensional vector representation (784-D), and AUDIO is a small-scale speech dataset.

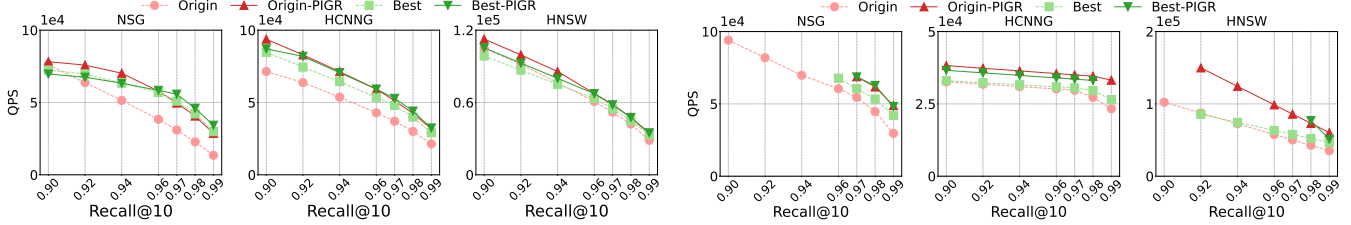
Algorithms. We use NSG, HCNNG, and HNSW as representative baselines, and run PIGR as a post-hoc refinement on each index, producing NSG+PIGR, HCNNG+PIGR, and HNSW+PIGR.



(a) Three index families on SIFT1M.

(b) NSG on multiple datasets.

Figure 5: Refinement dynamics (QPS–Recall@10): rapid early improvements followed by a late-stage plateau.



(a) SIFT1M

(b) MNIST

Figure 6: Comparison among Original, Best, and PIGR-refined graphs on two datasets (QPS–Recall@10).

Table 2: PIGR gains at high recall (Recall@10 = 0.99).

Index	SIFT1M	DEEP1M	DBPEDIA1M	AUDIO	MNIST	SIFT10M
NSG	2.11× / 43.8%	2.08× / 47.0%	2.58× / 60.7%	2.48× / 65.1%	1.54× / 24.4%	2.16× / 42.5%
HCNNG	1.41× / 18.4%	1.35× / 22.1%	1.72× / 46.1%	1.85× / 34.2%	1.41× / 5.4%	1.43× / 21.7%
HNSW	1.31× / 14.5%	1.37× / 23.4%	1.39× / 25.6%	1.87× / 43.0%	1.32× / 2.7%	1.33× / 18.3%

Note. Each cell reports QPS speedup / reduction of distance evaluations.

Metrics. We report the efficiency–accuracy trade-off as QPS vs. Recall@10 by default, focusing on the high-recall regime (Recall ≥ 0.9); results for larger k are reported when needed. In addition, we report the number of distance evaluations De (at Recall@10 = 0.99) as a hardware-agnostic proxy for query cost.

Parameters. The detailed index construction and PIGR default parameters are listed in Appendix H;

Implementation. All experiments are conducted on an Intel Xeon E5-2696 v4 @2.70GHz machine with 128GB RAM and 24 threads. Implementations are written in C++ and compiled with -Ofast.

5.2 Overall PIGR Performance

Answer to (Q1). Figure 4 shows that PIGR consistently improves the QPS–Recall@10 trade-off across all datasets and index families. Across the high-recall range (Recall@10 ≥ 0.9), the refined graphs deliver higher QPS at matched recall for NSG, HCNNG, and HNSW, with the largest gains concentrated near the near-exact recall. Similar gains are also observed for most Recall@ k settings with $k \in \{1, 50, 100\}$, suggesting that the benefits are not confined to Recall@10. Since the default refinement is configured with $k_{\text{core}} = 10$, these larger- k curves should be viewed as cross- k transfer results; occasional local drops, such as MNIST at moderate Recall@100, can occur when the refinement target and the evaluation k are mismatched. This behavior matches PIGR’s objective of reducing redundant distance evaluations under an unchanged search discipline.

To quantify the near-exact benefits, Table 2 reports matched-recall results at Recall@10 = 0.99 when feasible. At this operating point, PIGR substantially reduces De , translating into 1.3×–2.5× QPS speedups across the tested settings.

Robustness to shifted query workloads. Although refinement traces are collected only from base self-queries, the edge edits are applied to the graph topology and all external queries use the same refined

graph. We therefore evaluate the refined SIFT1M graphs on original, long-tail, and hard-query workloads disjoint from the base set. The gains remain stable across all three workloads, suggesting that the refinement captures structural routing improvements rather than memorizing a particular query set (see more details in Appendix F).

5.3 Stability and Convergence Analysis

Answer to (Q2). PIGR reaches a stable performance plateau within a few refinement iterations in the high-recall regime.

We visualize refinement dynamics using QPS–Recall@10 curves across iterations. Figure 5(a) tracks three index families on SIFT1M and shows the same stabilization trend across structurally distinct graphs. Figure 5(b) fixes the index family (NSG) and varies datasets; the behavior remains consistent across modalities and dimensionalities. Across the examined settings, the curves exhibit fast early improvement followed by a clear plateau: most gains are realized within the first 4–8 iterations, after which additional iterations yield only marginal changes in the high-recall region (see Appendix G for detailed stabilization statistics).

5.4 Comparison with Best-Tuned Baselines

Answer to (Q3). PIGR is complementary to build-time tuning. PIGR consistently outperforms the best-tuned baselines across different datasets; moreover, applying PIGR on top of the best-tuned construction yields further gains.

Best baselines via construction-parameter sweeps. Graph-based indexes can be sensitive to construction hyperparameters. We therefore perform construction-parameter sweeps for NSG, HCNNG, and HNSW on SIFT1M and MNIST to establish best tuned references. Full sweep curves are provided in Figure 9 (Appendix I).

Original vs. Best, with and without PIGR We start from two constructions: a typical setting (*Original*) and the best tuned setting selected from parameter sweeps (*Best*). For each, we report performance before and after applying PIGR, producing four curves: *Original*, *Original+PIGR*, *Best*, and *Best+PIGR*. As shown in Figure 6, *Original+PIGR* consistently outperforms the *Best Baseline* across the high-recall range. In addition, *Best+PIGR* further pushes

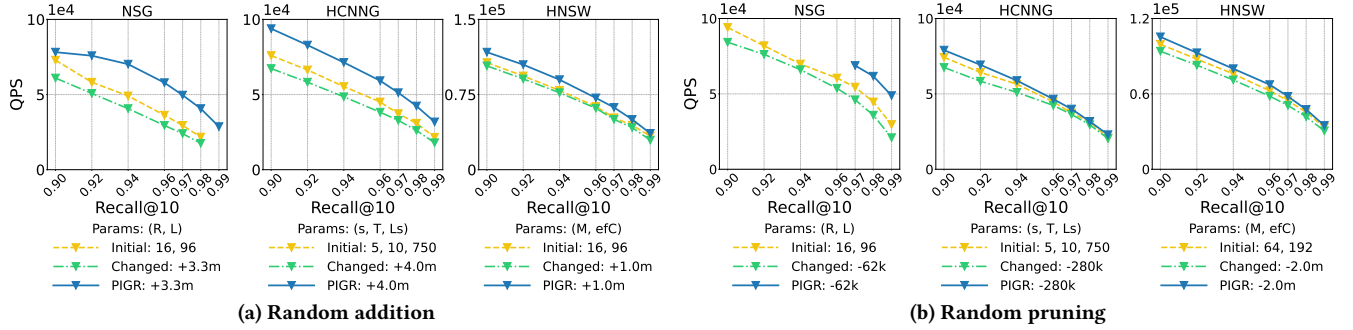


Figure 7: Random edge edits under matched budgets (QPS–Recall@10). For each setting, the random-edit budget is matched to the realized edge-count change $\Delta|E|$ of PIGR (Appendix Table 11); see Appendix J for more details.

Table 3: Relative changes w.r.t. the best-tuned baseline.

Dataset	Index	Variant(vs Best)	ΔE_n	ΔDe	$\Delta Size$	Time
SIFT1M	NSG	Orig+PIGR	↓38.2%	↓1.1%	↓37.8%	14.8×
		Best+PIGR	↑3.7%	↓7.5%	↑3.9%	8.9×
	HCNNG	Orig+PIGR	↑0.5%	↓5.4%	↑1.0%	10.2×
		Best+PIGR	↑13.8%	↓9.6%	↑10.7%	11.2×
	HNSW	Orig+PIGR	↓34.6%	↓9.7%	↓33.4%	8.3×
		Best+PIGR	↓5.2%	↓10.5%	↓9.4%	12.0×
MNIST	NSG	Orig+PIGR	↓43.3%	↓3.9%	↓42.2%	14.1×
		Best+PIGR	↓11.1%	↓3.7%	↓12.2%	6.8×
	HCNNG	Orig+PIGR	↑8.8%	↓3.8%	↑8.0%	13.6×
		Best+PIGR	↑12.2%	↓5.5%	↑11.5%	6.0×
	HNSW	Orig+PIGR	↓22.0%	↓1.1%	↓21.3%	10.2×
		Best+PIGR	↓21.0%	↓2.5%	↓17.2%	8.7×

Note. $\Delta M = (M_{\text{variant}} - M_{\text{Best}}) / M_{\text{Best}}$. Time is the total refinement time normalized by the best baseline’s build time. Absolute edit counts are reported in Appendix Table 11.

Table 4: Sampled self-queries on SIFT1M/NSG.

Sampling ratio	1.0	0.8	0.5	0.3
QPS↑/ Time	2.1× / 14.8×	2.0× / 11.0×	1.9× / 7.8×	1.6× / 5.7×

the Pareto frontier, indicating that post-hoc, trace-driven refinement captures efficiency opportunities beyond what construction-parameter search alone can exhaust.

To make this effect more interpretable at a matched operating point, Table 3 reports relative changes with respect to the *Best Baseline*. Across all six (dataset, index) settings, both *Original+PIGR* and *Best+PIGR* reduce *De*, which correspondingly translates into higher QPS at matched recall. Notably, these gains do not require systematically larger graphs: edge and memory footprints of *Original+PIGR* are comparable to the best-tuned baselines and are frequently smaller, particularly for NSG and HNSW. The refinement duration is typically around 5–15× the average construction time; however, this yields a growing time-cost advantage over grid search as the number of tunable construction parameters expands. If faster iterations are desired, one can reduce the number of trace samples collected per iteration; this reduces refinement time with a modest loss in speedup, as shown in Table 4.

Parameter choices. PIGR is not parameter-free. Its parameters, however, are operating-point or edit-budget choices on a fixed graph, rather than topology-inducing construction parameters that require repeated rebuilds. In our experiments we use stable defaults unless the target retrieval regime changes: k_{core} follows the target

Recall@ k , beam sizes are obtained by base-only calibration, and the pruning ratio is kept conservative. The ablations are intended as diagnostic guidance rather than a dataset-specific exhaustive search.

Robustness to construction parameters. We further examine the impact of varying construction parameters on DBPEDIA1M, which exhibits significant performance variance across different settings. Instead of an exhaustive sweep, we specifically evaluated PIGR on graphs initialized with several representative parameters commonly adopted in practice. As shown in Figure 8, PIGR consistently pushes the Pareto frontier upward across all tested parameters. This indicates that our refinement is robust to initialization differences and reliably unlocks efficiency gains without requiring the base index to be pre-tuned to a narrow optimal state.

5.5 Random Add and Prune

Answer to (Q4). A key concern is whether PIGR’s gains can be explained by simply changing the number of edges (e.g., merely increasing density), rather than by where the edits are made. We therefore construct random-edit controls under matched budgets to isolate the effect of targeted edits vs. arbitrary structural changes. The construction of the indexes is detailed in Appendix J.

Results and Analysis. As shown in Figure 7, random perturbations consistently degrade efficiency compared to the initial baseline (Yellow curves). **Random addition** (Green curves) lowers QPS because the computational cost of checking extra, untargeted neighbors (inflated degree) outweighs the benefit of incidental shortcuts. Conversely, **random pruning** causes sharp performance drops by severing critical small-world bridges essential for navigation. This confirms that PIGR’s gains stem not from simple densification or sparsification, but from *trace-aligned* edits that specifically create high-value highways and remove computational redundancies.

5.6 Additional Graph Families and Baselines

Answer to (Q5). The main experiments cover three representative graph families, while many recent ANNS graphs are variants or extensions of these designs. We therefore further test PIGR on Vamana and τ -MNG to examine recent graph variants. For post-construction edge editing, we include APG as the closest reproducible pruning baseline in our setting, since both APG and PIGR modify an already-built graph without changing the deployed search procedure. As summarized in Table 5, PIGR remains effective on these additional

Table 5: Additional graph families and baseline.

Group	Method / graph	Dataset	QPS↑ / Time
Post-edit	APG	SIFT1M	1.6× / 40.9×
	PIGR-NSG	SIFT1M	2.1× / 14.8×
	APG	DEEP1M	1.5× / 21.7×
	PIGR-NSG	DEEP1M	2.1× / 11.3×
Variant	Vamana+PIGR	SIFT1M	1.5× / 9.3×
	τ -MNG+PIGR	SIFT1M	1.4× / 13.4×

graph variants and compares favorably with APG under comparable index-construction parameters.

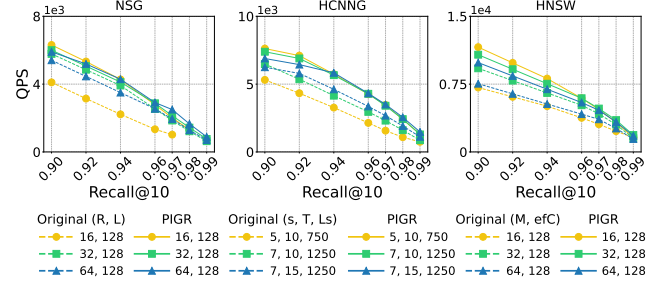
6 Related Work

Approximate nearest neighbor search. Approximate nearest neighbor search (ANNS) can be broadly categorized into four major methods: graph-based [7, 24, 31, 33], quantization-based [12, 17], tree-based methods [27, 30], and hashing-based [13, 15, 22]. Graph-based methods build proximity graphs and perform greedy beam search for retrieval, with representative designs such as HNSW [24], NSG [7], DiskANN/Vamana [31], and HCNNG [33], widely adopted in industry. Quantization methods reduce memory and computation costs [12, 17, 18]. Tree-based methods are typically effective in low-dimensional spaces [27, 30]. Hashing techniques offer guarantees for retrieving approximate neighbors [13, 15, 22]. Recent surveys and benchmarking studies provide comprehensive overviews of these techniques and their trade-offs [1, 36].

Post-construction edge editing. Beyond build-time construction, several works post-process or adapt graph indexes by pruning or adding edges. Closest to PIGR are GraSP and APG: GraSP estimates pruning impact using query-workload-driven subgraph sampling [38], while APG learns pruning decisions from a dense graph [39]. In contrast, PIGR uses base-only self-queries, combines trace-guided pruning with shortcut insertion, and does not require the deployment query workload; we include reproduced APG comparisons in Section 5.6. DNG moves pruning into the search procedure via low-dimensional representations and direction-guided online pruning [34]. Complementary to pruning, Liu *et al.* add reverse connections for low-degree vertices and use model-based control to adapt traversal budgets [21]. Wolverine repairs monotone decreasing search paths for dynamic index updates, whereas PIGR targets static post-hoc refinement of an already-built graph [40]. Vista studies distribution-aware vector indexing and search for large-scale imbalanced datasets, whereas PIGR refines an already-built graph from self-query traces; we therefore regard Vista as complementary rather than a direct post-construction edge-editing baseline [8]. EnhanceGraph continuously exploits search and construction logs through an auxiliary conjugate graph, while PIGR uses base-only self-query traces to edit the original graph without changing the deployed search kernel, so we do not treat it as the same directly comparable plug-in setting [41].

7 Conclusion

In this paper, we revisit graph-based ANNS by moving beyond structure-driven tuning to trace-driven post-construction refinement for high-recall efficiency. Following a cost-aligned principle, we develop PIGR, a self-query-driven, trace-guided framework that edits the edge set based on objective traversal evidence—pruning

**Figure 8: Effect of PIGR across multiple initial parameter configurations on DBPEDIA1M (QPS–Recall@10).**

persistently low-yield edges and inserting targeted shortcuts to mitigate detour-driven redundancy—while keeping the query procedure unchanged. Across diverse graph families and datasets, experiments consistently show that this trace-guided edge editing improves the efficiency–accuracy trade-off in the high-recall regime.

LLM Usage Description

We used a large language model (LLM) solely as a language-polishing tool to improve writing clarity and presentation (e.g., grammar, wording, and readability). The authors are solely responsible for the correctness and authenticity of all experimental results and claims reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (No. 62572126) and the Key Projects of the National Natural Science Foundation of China (No. U23A20496).

References

- [1] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANNS-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374.
- [2] Franz Aurenhammer. 1991. Voronoi diagrams—a survey of a fundamental geometric data structure. *ACM computing surveys (CSUR)* 23, 3 (1991), 345–405.
- [3] Yun Chen, Xueqi Cheng, Hui Liu, Yushuang Wu, and Yue Wang. 2023. FIN-GER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference (WWW)*. doi:10.1145/3543507.3583318
- [4] Thomas Cover and Peter Hart. 1967. Nearest neighbor pattern classification. *IEEE transactions on information theory* 13, 1 (1967), 21–27.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.
- [6] Magdalen Dobson, Zheqi Shen, Guy E Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2023. Scaling graph-based anns algorithms to billion-size datasets: A comparative analysis. *arXiv preprint arXiv:2305.04359* (2023).
- [7] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474. doi:10.14778/3303753.3303754
- [8] Yujian Fu, Cheng Chen, Yao Chen, Weng-Fai Wong, and Bingsheng He. 2025. Vista: Vector Indexing and Search for Large-scale Imbalanced Datasets. In *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE)*. 543–556.
- [9] Keinosuke Fukunaga and Patrenahalli M. Narendra. 2006. A branch and bound algorithm for computing k-nearest neighbors. *IEEE transactions on computers* 100, 7 (2006), 750–753.
- [10] Jianyang Gao and Cheng Long. 2023. High-dimensional approximate nearest neighbor search: with reliable and efficient distance comparison operations. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–27.
- [11] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented Generation for Large Language Models: A Survey. *arXiv:2312.10997 [cs.CL]*

- [12] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2946–2953. https://openaccess.thecvf.com/content_cvpr_2013/papers/Ge_Optimized_Product_Quantization_2013_CVPR_paper.pdf
- [13] Aristides Gionis, Piotr Indyk, Rajeev Motwani, et al. 1999. Similarity search in high dimensions via hashing. In *Vldb*, Vol. 99. 518–529.
- [14] Silu Huang, Ada Wai-Chee Fu, and Ruifeng Liu. 2015. Minimum spanning trees in temporal graphs. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 419–430.
- [15] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing (STOC)*. 604–613. doi:10.1145/276698.276876
- [16] Jerzy W Jaromczyk and Godfried T Toussaint. 2002. Relative neighborhood graphs and their relatives. *Proc. IEEE* 80, 9 (2002), 1502–1517.
- [17] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57
- [18] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547. doi:10.1109/TBDDATA.2019.2921572
- [19] Atsutake Kosuge and Takashi Oshima. 2019. An object-pose estimation acceleration technique for picking robot applications by using graph-reusing k-nn search. In *2019 First International Conference on Graph Computing (GC)*. IEEE, 68–74.
- [20] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2019. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2019), 1475–1488.
- [21] Jun Liu, Zhenhua Zhu, Jingbo Hu, Hanbo Sun, Li Liu, Lingzhi Liu, Guohao Dai, Huazhong Yang, and Yu Wang. 2022. Optimizing Graph-based Approximate Nearest Neighbor Search: Stronger and Smarter. In *23rd IEEE International Conference on Mobile Data Management (MDM)*. 179–184. doi:10.1109/MDM55031.2022.00045
- [22] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2007. Multi-Probe LSH: Efficient Indexing for High-Dimensional Similarity Search. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB)*. 950–961. <https://www.vldb.org/conf/2007/papers/research/p950-lv.pdf>
- [23] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.
- [24] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473
- [25] Ben Mann, Nick Ryder, Melanie Subbiah, J Kaplan, P Dhariwal, A Neelakantan, P Shyam, G Sastry, A Askell, S Agarwal, et al. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* 1, 3 (2020), 3.
- [26] Yitong Meng, Xinyan Dai, Xiao Yan, James Cheng, Weiwen Liu, Jun Guo, Benben Liao, and Guangyong Chen. 2020. Pmd: An optimal transportation-based user distance for recommender systems. In *European Conference on Information Retrieval*. Springer, 272–280.
- [27] Marius Muja and David G. Lowe. 2014. Scalable Nearest Neighbor Algorithms for High Dimensional Data. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 11 (2014), 2227–2240. doi:10.1109/TPAMI.2014.2321376
- [28] Flickner Myron. 1995. Query by image and video content: The QBIC system. *IEEE computer* (1995), 23–32.
- [29] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*. 285–295.
- [30] Tanachai Silpa-Anan and Richard Hartley. 2008. Optimised KD-Trees for Fast Image Descriptor Matching. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. doi:10.1109/CVPR.2008.4587638
- [31] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [32] Godfried T. Toussaint. 1980. The Relative Neighbourhood Graph of a Finite Planar Set. *Pattern Recognition* 12, 4 (1980), 261–268. doi:10.1016/0031-3203(80)90066-7
- [33] J. E. Vargas Muñoz, M. A. González, Z. Dias, and R. da S. Torres. 2019. Hierarchical clustering-based graphs for large scale approximate nearest neighbor search. *Pattern Recognition* 96 (2019), 106970. doi:10.1016/j.patcog.2019.106970
- [34] Gang Wang, Xingxing Ke, Jian Hu, Qianling Li, Minghao Shao, and Jianfeng Fan. 2022. Learning to Prune: General and Efficient Approximate Nearest Neighbor Search with Direction Navigating Graph. In *Proceedings of the 2022 5th International Conference on Data Storage and Data Engineering (DSDE)*. 50–58. doi:10.1145/3528114.3528123
- [35] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 international conference on management of data*. 2614–2627.
- [36] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 14, 11 (2021), 1964–1978. doi:10.14778/3476249.3476255
- [37] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data. *Proc. VLDB Endow.* 13, 12 (2020), 3152–3165.
- [38] Fei Zhang, Hui Liu, Xueqi Cheng, Yushuang Wu, and Yue Wang. 2022. GraSP: Optimizing Graph-based Nearest Neighbor Search with Subgraph Sampling and Pruning. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining (WSDM)*. 1395–1405. doi:10.1145/3488560.3498470
- [39] Minjia Zhang, Wenhan Wang, and Yuxiong He. 2020. Learning to Anneal and Prune Proximity Graphs for Similarity Search. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=HJlXC3EtwB>
- [40] Xiao Zheng, Haocheng Han, Liuhaio Zhang, Chao Chen, Shuangjie Peng, Ke Sun, and Yi Zhang. 2025. Wolverine: Highly Efficient Monotonic Search Path Repair for Graph-based ANN Index Updates. *Proceedings of the VLDB Endowment* 18, 9 (2025), 2529–2542. doi:10.14778/3740412.3740430
- [41] Xiaoyao Zhong, Jiabao Jin, Peng Cheng, Mingyu Yang, Lei Chen, Haoyang Li, Zhitao Shen, Xuemin Lin, Heng Tao Shen, and Jingkuan Song. 2025. EnhanceGraph: A Continuously Enhanced Graph-based Index for High-dimensional Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2506.13144* (2025).
- [42] Chun Jiang Zhu, Tan Zhu, Haining Li, Jinbo Bi, and Minghu Song. 2019. Accelerating large-scale molecular similarity search through exploiting high performance computing. In *2019 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, 330–333.

Extended Version Statement

This document is an extended version of the KDD 2026 camera-ready paper. It includes additional derivations, implementation details, full construction sweeps, ablation studies, and robustness checks that are omitted from the proceedings version due to page limits.

A Greedy Beam Search

We summarize greedy beam search in Algorithm 4. Index-specific components (e.g., entry-point selection, hierarchical descent, or guided neighbor filtering [7, 24, 33]) affect initialization or admissible neighbor expansions, while the core best-first expansion below remains the same.

Algorithm 4 Greedy beam search (Core Expansion)(G, q, k, ef)

Require: Index G , query q , neighbor count k , beam size ef

Ensure: Top- k closest elements found in H_R

- 1: Select entry point ep in G ; Initialize H_C (min-heap), H_R (max-heap)
 - 2: Insert ep into H_C, H_R ; Mark ep as visited
 - 3: **while** H_C is not empty:
 - 4: $c \leftarrow H_C.\text{popMin}()$
 - 5: $d_{\max} \leftarrow$ distance of farthest node in H_R to q (if H_R full, else ∞)
 - 6: **if** $|H_R| \geq ef$ **and** $\text{dis}(c, q) > d_{\max}$:
 - 7: **break**
 - 8: **for** each unvisited neighbor $v \in N(c)$ passing filters:
 - 9: Mark v visited; $d \leftarrow \text{dis}(v, q)$
 - 10: **if** $|H_R| < ef$ **or** $d < d_{\max}$:
 - 11: $H_C.\text{push}(v)$; $H_R.\text{push}(v)$
 - 12: **if** $|H_R| > ef$:
 - 13: $H_R.\text{popMax}()$; Update $d_{\max}$
 - 14: **return** Top- k elements in H_R
-

B Conservative Analysis for Low-EER-First Pruning

We now give a conservative argument for why pruning low-EER edges tends to reduce the expected number of distance evaluations when recall is preserved. Let $G = (V, E)$ be the original graph and $G' = (V, E \setminus S)$ the graph obtained by pruning an edge set $S \subseteq E$. For a query q , $\text{De}(G, q) = \text{adopt}(G, q) + \text{discard}(G, q)$, denotes the total number of distance evaluations performed on G , where an evaluation is *adopted* if it inserts the evaluated neighbor into the bounded best list H_R (tightening the current H_R threshold) at the time of evaluation, and *discarded* otherwise.

For each edge $e \in E$ and query q , let $I_e(q) \in \{0, 1\}$ be the indicator that e triggers a distance evaluation in the search trace of q on G . Under a fixed search procedure that expands each node at most once, each directed edge is evaluated at most once per query, so $I_e(q)$ captures whether e participates in $\text{De}(G, q)$. Conditioning on a single evaluation event through $e \in S$, pruning e always removes that original distance evaluation, but may incur additional distance evaluations when the removed evaluation would have been adopted. We proceed under the following mild conditions, which are used to explain the pruning tendency rather than to provide an unconditional guarantee:

- (A1) **Fixed search discipline.** The search procedure and parameters (entry policy, beam size, queue policy, and tie-breaking) are fixed.
- (A2) **Recall preservation (conditioned).** The pruning step under consideration preserves the fixed-recall target on the workload: $\text{Rec}_K(G'; Q) \geq \tau$.
- (A3) **Bounded replacement overhead (in expectation).** If an evaluation through e in G would have been adopted, the search on G' recovers an alternative adopted event with an expected extra cost of at most $\bar{c}_e \geq 0$ distance evaluations (typically small in practice), and $\bar{c}_e \leq c_{\max}$.

Here $c_{\max}$ is introduced only as a conceptual upper bound to obtain a conservative trend; our method does not estimate or tune $c_{\max}$ in practice. Conditioning on a single evaluation event through $e \in S$, we distinguish two cases:

- (1) If the event would have been *discarded*, pruning saves one distance evaluation, so the change in distance evaluations is -1 .
- (2) If the event would have been *adopted*, pruning removes this distance evaluation, but the search on G' may need to recover an alternative adopted event. By (A2)–(A3), the expected net change is at most $-1 + \bar{c}_e$.

Let $p_e = \Pr(\text{adopt} \mid \text{evaluate via } e)$. Taking expectations over the adopt/discard outcome for a single evaluation gives

$$\mathbb{E}[\Delta_e^{\text{per-eval}}] \leq (1 - p_e) \cdot (-1) + p_e \cdot (-1 + \bar{c}_e) = -1 + \bar{c}_e p_e,$$

where $\Delta_e^{\text{per-eval}}$ is the change in distance evaluations contributed by that evaluation.

Now consider the effect over the query workload Q . Summing the per-evaluation contributions gives the total change. While pruning multiple edges could theoretically induce coupled rerouting effects (where removing one edge drastically alters the traffic on another), such *second-order interactions* are negligible when the pruning set S is small relative to $|E|$. Under this **small perturbation regime**,

we approximate the total change linearly:

$$\mathbb{E}_q[\text{De}(G', q) - \text{De}(G, q)] \approx \sum_{e \in S} \mathbb{E}_q[I_e(q) \Delta_e^{\text{per-eval}}(q)].$$

where $\Delta_e^{\text{per-eval}}(q)$ equals $\Delta_e^{\text{per-eval}}$ when e is evaluated and 0 otherwise. Taking expectations over $q \sim Q$ and using linearity of expectation,

$$\mathbb{E}_q[\text{De}(G', q) - \text{De}(G, q)] = \sum_{e \in S} \mathbb{E}_q[I_e(q) \Delta_e^{\text{per-eval}}(q)].$$

Let $\mu_e = \Pr_q(\text{evaluate via } e) = \mathbb{E}_q[I_e(q)]$ denote the probability that e is evaluated by a random query from Q . (Equivalently, if $\text{vis}(e)$ counts evaluation events over the trace set, then $\mu_e \approx \text{vis}(e)/|Q|$.) Using the per-evaluation bound and $\bar{c}_e \leq c_{\max}$, we obtain

$$\mathbb{E}_q[I_e(q) \Delta_e^{\text{per-eval}}(q)] \leq \mu_e (-1 + \bar{c}_e p_e) \leq \mu_e (-1 + c_{\max} p_e),$$

and hence

$$\mathbb{E}_q[\text{De}(G', q) - \text{De}(G, q)] \leq \sum_{e \in S} \mu_e (-1 + c_{\max} p_e).$$

Equivalently, the expected *reduction* in distance evaluations satisfies

$$\mathbb{E}_q[\text{De}(G, q) - \text{De}(G', q)] \geq \sum_{e \in S} \mu_e (1 - c_{\max} p_e).$$

This motivates the following conservative lower bound on the expected gain contributed by pruning a single edge e :

$$\text{gain}_{\text{lb}}(e) = \mu_e (1 - c_{\max} p_e).$$

Replacing p_e by its empirical estimate $\text{EER}(e)$ gives

$$\text{gain}_{\text{lb}}(e) \approx \mu_e (1 - c_{\max} \text{EER}(e)).$$

In particular, whenever $\text{EER}(e) \leq 1/c_{\max}$, the bound suggests that pruning e is unlikely to increase the expected number of distance evaluations; at $\text{EER}(e) = 0$, we have $\text{gain}_{\text{lb}}(e) = \mu_e > 0$ if e is ever evaluated, so pruning is strictly beneficial in this extreme case.

For fixed visit probability μ_e , $\text{gain}_{\text{lb}}(e)$ decreases monotonically in $\text{EER}(e)$. Thus, among sufficiently supported edges, smaller $\text{EER}(e)$ implies a larger conservative lower bound on expected savings. We emphasize that this analysis explains a robust tendency rather than a universal guarantee: the realized gains may vary due to correlations in how edges are evaluated and utilized during search.

C A Sketch Intuition for Why Bridging Saves Distance Evaluations

Distance-evaluation-level intuition. We sketch an intuition (not a guarantee) for why well-chosen bridges can reduce the number of distance evaluations under a fixed recall target. Consider a graph $G = (V, E)$ searched by an expansion-based traversal. For a query q , suppose the query-adjacent neighborhood that must be explored to reach recall τ splits into $m \geq 2$ locally dense components $\{R_1, \dots, R_m\}$ that are only weakly connected by *effective* cross-component links for the deployed traversal. Let the expansion order induce depth indices $\ell = 0, 1, \dots$, and let c_ℓ denote the expected number of distance evaluations incurred around depth

ℓ . Moving from one component to another via backtracking then incurs an expected detour cost on the order of

$$s_{\min} \approx \sum_{j=1}^{h-1} c_{\ell-j},$$

where $h \geq 2$ is the typical height of such backtracking segments.

Now suppose we add bridging edges (u, v) with $u \in R_i, v \in R_j$ ($i \neq j$) such that $\text{dis}(v, q)$ is substantially smaller than $\text{dis}(u, q)$. When u is expanded, evaluating (u, v) can expose v early and allow the traversal to hand over to R_j near depth ℓ , skipping the detour segment. Let S_e be the (random) number of distance evaluations saved by skipping this detour and X_e the additional evaluations introduced by evaluating and possibly expanding v . Then the net change attributable to $e = (u, v)$ is $Y_e = S_e - X_e$. A simple decomposition yields

$$\mathbb{E}[S_e] \geq \pi_e s_{\min}, \quad \mathbb{E}[X_e] \leq b_{\max},$$

where $\pi_e > 0$ is the probability that the bridge effectively triggers such an early handover, and $b_{\max}$ upper-bounds the expected overhead of the bridge (e.g., one extra distance evaluation plus a few local expansions). Consequently,

$$\mathbb{E}[Y_e] \geq \pi_e s_{\min} - b_{\max}.$$

Thus, a bridge tends to yield net savings when $\pi_e s_{\min} > b_{\max}$: the larger the detour cost and the smaller the overhead, the more likely the bridge is beneficial.

D Beam Size Selection for Refinement

To avoid relying on any external query workload, we choose the beam sizes used in PIGR via a base-only calibration set. Concretely, we uniformly sample a small subset $Q_{\text{cal}} \subset S$ (e.g., 1% of vertices) as calibration queries and compute their ground-truth k -NNs within the base set once [9]. We reuse the same Q_{cal} throughout all refinement rounds.

After each refinement iteration updates the graph, we run the deployed search on Q_{cal} and map recall targets to beam sizes:

- (1) **Probe size ef_{gc} .** We set ef_{gc} to the smallest beam size such that the traversal achieves $\text{Rec}_k(G; Q_{\text{cal}}) \geq 0.99$. This provides a broader probe that is more likely to expose nearby but under-reached regions when proposing shortcut candidates in the add module.
- (2) **Target size ef_{target} .** We set ef_{target} to the smallest beam size such that $\text{Rec}_k(G; Q_{\text{cal}}) \geq \tau$ for a user-specified target τ (e.g., 0.9). This size characterizes the neighborhood that the target small-beam traversal can reliably reach, and thus the region whose reachability PIGR aims to improve under the deployed budget.

In addition, we set the pruning trace size to $ef_{\text{prune}} = ef_{\text{gc}}$ in our implementation. Using the same broader probe for pruning yields wider trace coverage and more stable empirical $\text{EER}(\cdot)$ estimates, while still preserving the fixed traversal discipline (only the beam size is swept for trace collection). Unless otherwise stated, we also set $ef_{\text{add}} = ef_{\text{gc}}$ for the add module to avoid extra tuning.

This calibration procedure only determines the internal query budgets used by PIGR during refinement; it does not change the deployed traversal used for evaluation.

Using the evaluation query set as a practical alternative. While we default to a base-only calibration set $Q_{\text{cal}} \subset S$ to avoid any dependence on an external workload, in practice one may also determine ef_{gc} and ef_{target} using the evaluation query set Q_{test} . This does not affect the refinement itself because Q_{test} is never used as self-queries to collect traces or to propose/score edge edits; it is only used to *measure* the performance of the updated index after each iteration. Moreover, the add module only requires a clear separation between the probe size and the deployed size, i.e., choosing $ef_{\text{target}} < ef_{\text{gc}}$ is sufficient to ensure that the large-beam probe exposes broader neighborhoods than the small-beam traversal. Therefore, if one prefers not to sample an extra calibration subset, Q_{test} can serve as a convenient surrogate for mapping recall targets to beam sizes.

E Complexity Details

We provide detailed cost accounting for one PIGR refinement round under the deployed greedy beam search kernel. Let $N = |V|$ be the number of vertices and $|E|$ the number of directed edges. For beam size ef , $\text{De}_{ef}(q)$ denotes the number of distance evaluations performed by greedy beam search for query q on the current graph. We also define $ef_{\text{max}} = \max\{ef_{\text{prune}}, ef_{\text{gc}}, ef_{\text{target}}, ef_{\text{add}}\}$.

A standard implementation maintains a bounded candidate structure (beam / bounded best list) of size $O(ef)$, typically using a heap or ordered container. This contributes an extra $O(\log ef)$ overhead per distance evaluation. Thus, a run that performs $\text{De}_{ef}(q)$ distance evaluations costs $O(\text{De}_{ef}(q) \log ef)$ time in addition to vector-distance arithmetic. When distance evaluations dominate, we primarily report costs in distance-evaluation counts and include the $\log ef$ factor when stating time bounds.

Pruning (Algorithm 1). Pruning collects traces by running greedy beam search with size ef_{prune} for every self-query $q \in S$. Hence the distance-evaluation cost is

$$D_{\text{prune}} = \sum_{q \in S} \text{De}_{ef_{\text{prune}}}(q),$$

and the corresponding time is $O(D_{\text{prune}} \log ef_{\text{prune}})$ under the heap-based maintenance note above.

After trace collection, only directed edges that appear in traces are ranked. Let $E_{\text{act}} \subseteq E$ be the set of edges with $\text{adopt}(e) + \text{discard}(e) > 0$. Computing scores is linear in $|E_{\text{act}}|$, and sorting (or selecting top- B) costs $O(|E_{\text{act}}| \log |E_{\text{act}}|)$ in the worst case.

Calibration traces (Algorithm 2). For each $v \in S$, calibration runs two greedy-beam searches: one with size ef_{gc} to obtain $\text{HCN}(v)$ and one with size ef_{target} to obtain $\text{LCN}(v)$. Thus the distance-evaluation cost is

$$D_{\text{cal-trace}} = \sum_{v \in S} (\text{De}_{ef_{\text{gc}}}(v) + \text{De}_{ef_{\text{target}}}(v)),$$

and the corresponding time is

$$O\left(\sum_{v \in S} (\text{De}_{ef_{\text{gc}}}(v) \log ef_{\text{gc}} + \text{De}_{ef_{\text{target}}}(v) \log ef_{\text{target}})\right).$$

Calibration attachment selection (extra distance evaluations). Calibration adds attachment links for nodes in $M(v) = \text{HCN}(v) \setminus \text{LCN}(v)$, anchored from the overlap $O(v) = \text{HCN}(v) \cap \text{LCN}(v)$. For each $w \in M(v)$, selecting $u^*(w) = \arg \min_{u \in O(v)} \text{dis}(u, w)$ requires

$|O(v)|$ distance computations in the simplest exhaustive implementation. Therefore, the extra distance-evaluation count is

$$D_{\text{attach}} = \sum_{v \in S} \sum_{w \in M(v)} |O(v)|.$$

Since $|HCN(v)| = |LCN(v)| = k$, we have $|M(v)| \leq k$ and $|O(v)| \leq k$, yielding the bound $D_{\text{attach}} \leq |S| \cdot k^2$. If the implementation caps the number of attachments per v by a small constant b_{gc} , then $D_{\text{attach}} \leq |S| \cdot b_{\text{gc}} \cdot k$.

Bridging trace collection (Algorithm 3). For each seed $u \in \mathcal{U}$, bridging runs greedy beam search with size ef_{add} to collect trace information and form $R(u)$. Thus the distance-evaluation cost is

$$D_{\text{bridge-trace}} = \sum_{u \in \mathcal{U}} D_{e_{f_{\text{add}}}}(u),$$

and the corresponding time is $O(D_{\text{bridge-trace}} \log ef_{\text{add}})$.

Bridging local connectivity updates (non-distance overhead). On each $R(u)$ of size k , Algorithm 3 builds a trace-induced directed graph and computes J -hop connectivity on it once, followed by a constant number of edge insertions (up to two directed edges). A conservative worst-case bound for naive matrix-style reachability computation is $O(J \cdot k^3)$ per seed (plus $O(k^2)$ for identifying the weakest-connected node and its best partner), hence $O(|\mathcal{U}| \cdot J \cdot k^3)$ overall. In typical settings, k and J are small constants, so this term is usually lower-order compared with global distance evaluations.

Putting everything together (one round). Combining the above, the total number of distance evaluations per round is

$$D_{\text{round}} = D_{\text{prune}} + D_{\text{eal-trace}} + D_{\text{bridge-trace}} + D_{\text{attach}},$$

. The corresponding time bound (including the optional heap-maintenance factor) is

$$O(D_{\text{round}} \log ef_{\text{max}} + |E_{\text{act}}| \log |E_{\text{act}}| + |\mathcal{U}| \cdot J \cdot k^3).$$

Interpretation (constant-number-of-sweeps intuition). When $S = V$ and $\mathcal{U}$ is a subset of vertices, one refinement round performs a constant number of greedy-beam passes over base points (with sizes ef_{prune} , ef_{gc} , and ef_{target}) plus one pass over $\mathcal{U}$ (with size ef_{add}), together with a lightweight $O(|S|k^2)$ attachment-selection term. This makes the refinement overhead comparable to a small constant number of full base sweeps under the same search kernel.

F Additional Robustness Checks

Shifted query workloads. Although PIGR uses base points as self-queries during refinement, the refined graph is evaluated with query workloads disjoint from the base set. To further check whether the improvement depends on a particular query distribution, we evaluate the same refined SIFT1M graphs under original, long-tail, and hard-query workloads. As shown in Table 6, the gains remain stable across all three index families.

Table 6: Robustness to shifted SIFT1M query workloads.

Index	Original	Long-tail	Hard
NSG+PIGR	2.1×	1.9×	2.0×
HCNNG+PIGR	1.4×	1.4×	1.5×
HNSW+PIGR	1.3×	1.4×	1.3×

Table 7: MNIST Recall@100 under target- k refinement. The refinement is run with $k_{\text{core}} = 100$; each column reports QPS at the matched Recall@100 operating point.

Recall@100	0.90	0.92	0.94	0.96	0.97	0.98	0.99
Base QPS	30,993	30,120	28,447	24,346	22,295	19,555	14,428
PIGR QPS	31,579	30,896	30,213	28,543	27,483	26,423	23,829
Speedup	1.02×	1.03×	1.06×	1.17×	1.23×	1.35×	1.65×

Target- k refinement on MNIST Recall@100. As discussed in Section 5.2, Figure 4 reports Recall@100 under the default refinement setting $k_{\text{core}} = 10$. Therefore, the MNIST Recall@100 curve is a cross- k transfer result rather than a refinement specifically optimized for retrieving 100 neighbors. To isolate this target- k mismatch, we rerun PIGR on MNIST with the returned-neighbor scope set to $k_{\text{core}} = 100$, while keeping the deployed search kernel and evaluation protocol unchanged. As shown in Table 7, target- k refinement improves QPS at all matched Recall@100 operating points, with gains increasing from 1.02×

at Recall@100 = 0.90 to 1.65×

G Additional Convergence Statistics

We provide additional convergence statistics in Table 8 and Table 9.

Table 8: Late-stage QPS changes on SIFT1M.

	NSG	HCNNG	HNSW
SIFT1M	1.7% / 0.4%	0.9% / -1.1%	2.9% / -2.2%

Note. Each entry reports $\Delta_{10 \rightarrow 8} / \Delta_{12 \rightarrow 10}$. For two rounds t_2 and t_1 , $\Delta_{t_2 \rightarrow t_1}$ is the mean relative QPS change of round t_2 relative to t_1 , evaluated at Recall@10 points $\mathcal{R} = \{0.96, 0.97, 0.98, 0.99\}$ (QPS obtained by linear interpolation on each QPS–Recall curve): $\Delta_{t_2 \rightarrow t_1} = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} \frac{QPS_{t_2}(r) - QPS_{t_1}(r)}{QPS_{t_1}(r)}$.

Table 9: Late-stage QPS changes across datasets (NSG).

	DEEP1M	DBPEDIA1M	AUDIO	MNIST
NSG	8.9% / 2.3%	3.1% / 1.0%	2.7% / -2.1%	1.2% / -1.9%

Note. Each entry reports $\Delta_{8 \rightarrow 6} / \Delta_{10 \rightarrow 8}$ using the same definition of $\Delta_{t_2 \rightarrow t_1}$ as in Table 8, evaluated at Recall@10 points $\mathcal{R} = \{0.96, 0.97, 0.98, 0.99\}$ (QPS by linear interpolation on each QPS–Recall curve).

H Experimental Settings

Parameters of constructing index are shown as follows (default settings unless otherwise stated):

Table 10: Dataset statistics and types.

Dataset	Dimension	Base	Query	Type
SIFT1M	128	1,000,000	10,000	Image
DEEP1M	96	990,000	10,000	Image
DBPEDIA1M	1536	990,000	10,000	Text
MNIST	784	60,000	10,000	Image
AUDIO	192	53,387	200	Audio
SIFT10M	128	10,000,000	10,000	Image

- NSG. The degree upper bound $R = 16$ and the $L = 96$ is the beam size.
- HCNNG. The number of hierarchical clustering trees $T = 10$, the leaf size of MST $L_s = 750$, and the MST degree $s = 5$.
- HNSW. The degree upper bound $M = 16$ and the $efC = 96$ is the upper bound of constructing candidate set size.

These parameters are from prior studies[1], with minor implementation level deviations that remain within these standard ranges. To account for construction sensitivity, we additionally perform grid search for *all three* indexes on selected datasets and report the best-performing configurations as *Best Baselines* in subsection 5.4. **Parameters of searching and Refinements.** The upper bound of the search beam size ef is set to range from 10 to 800. And the maximum number of iteration of PIGR is 20. The prune ratio $\rho = 0.02$, recall target $recall_{target} = 0.9$ and the k of addition $k_{core} = 10$. The $recall$ for ef_{gc} is fixed at 0.99, and we set $ef_{gc} = ef_{prune}$. In the add module, the calibration stage imposes no explicit limit on the number of added edges, whereas the bridging stage adds one to two edges per iteration.

I Full Construction-Parameter Sweeps

The full construction parameter sweeps serves as the reference to best-tuned baseline shown in Figure 9.

J Edge-Edit Budgets for Matched Random Baselines

This appendix reports the absolute edge-edit counts of PIGR and the realized net edge-count change $\Delta|E|$ (directed edges). These values (in Table 11) are used to set the matched-budget random-edit baselines in Figure 7. For each (dataset, index, variant), we define the matched edit budget as the realized net change in the number of directed edges, $\Delta|E| = |E_{after}| - |E_{before}|$, computed from the measured edge counts before and after refinement.

Matched-budget construction rules. Given $\Delta|E|$ for a (dataset, index, variant) setting, we instantiate random-edit controls while keeping the query procedure unchanged. If $\Delta|E| > 0$, *Random addition* adds exactly $|\Delta|E|$ directed edges by uniformly sampling absent directed pairs. If $\Delta|E| < 0$, *Random pruning* removes exactly $|\Delta|E|$ directed edges by uniformly sampling existing directed edges. Each random-edit control is evaluated under the same ef sweep as the main experiments.

Special case: HCNNG pruning control on SIFT1M. For HCNNG on SIFT1M, PIGR yields $\Delta|E| > 0$ in all reported settings (Table 11),

and thus there is no net edge-decrease budget to instantiate a $\Delta|E|$ -matched *random-pruning* control. To still assess whether indiscriminate pruning alone could explain the observed gains, we add an auxiliary pruning control on the *ordinary* HCNNG(SIFT1M) baseline: we prune one iteration using the same pruning ratio as PIGR’s prune step, record the number of removed directed edges, and then perform uniform-at-random edge removals under the same removal count. This provides a pruning comparison at a realistic one-iteration pruning scale without introducing an edge-decrease budget that does not occur in PIGR for this setting.

Table 11: Absolute edge-edit counts corresponding to Table 3. For each (dataset, index, variant), we report the number of pruned edges (Cut) and added edges in the two add stages (PGT_add and Matrix_add), together with the realized net change in directed edges $\Delta|E|$.

Dataset	Index	Variant	Cut	B_add	C_add	$\Delta E $
SIFT1M	NSG	Original+PIGR	1245648	1369902	3155725	+3279979
		Best+PIGR	695154	409246	1071773	+785865
	HCNNG	Original+PIGR	961211	959745	3758713	+3757247
		Best+PIGR	1152322	630231	3004550	+2482459
	HNSW	Original+PIGR	868899	446998	1285605	+863054
		Best+PIGR	3887876	402220	1792374	-1692982
MNIST	NSG	Original+PIGR	215348	24112	128830	-62406
		Best+PIGR	133729	16206	45508	-72015
	HCNNG	Original+PIGR	95527	45176	134681	+84330
		Best+PIGR	27138	35571	57004	+65437
	HNSW	Original+PIGR	254682	19448	194018	-41216
		Best+PIGR	281040	9618	75366	-196056

Note. $\Delta|E| = |E_{after}| - |E_{before}|$ denotes the realized net change in the number of directed edges (computed from the measured edge counts), which complements the relative edge-count change ΔE_n reported in Table 3.

K Parameter Choices and Diagnostic Ablations

This appendix clarifies the refinement-side choices of PIGR and their practical tuning burden. We do not claim that PIGR is parameter-free. The intended distinction is that construction-time tuning repeatedly rebuilds graphs under coupled, topology-inducing parameters, whereas PIGR edits an already-built graph and keeps the deployed search kernel unchanged.

The parameters involved in PIGR fall into three groups. First, the beam sizes used for trace collection, including ef_{gc} , ef_{target} , and ef_{prune} , are obtained by the base-only calibration procedure; they are not selected by an external query-workload grid search. Second, the hop parameter J is kept as a small local-reachability constant in the bridging implementation and is not tuned per dataset in our reported experiments. Third, the remaining refinement-side choices have direct operational meanings: the pruning ratio ρ controls the per-round edit budget, $recall_{target}$ controls the trace operating point for the add module, and k_{core} controls the retrieval scope used for local bridging.

The following ablations are therefore diagnostic rather than a required deployment-time exhaustive scan. They show that stable defaults arise from the mechanisms of the refinement loop: conservative pruning, a moderately faithful target trace, and a retrieval-scope choice tied to the target Recall@ k regime.

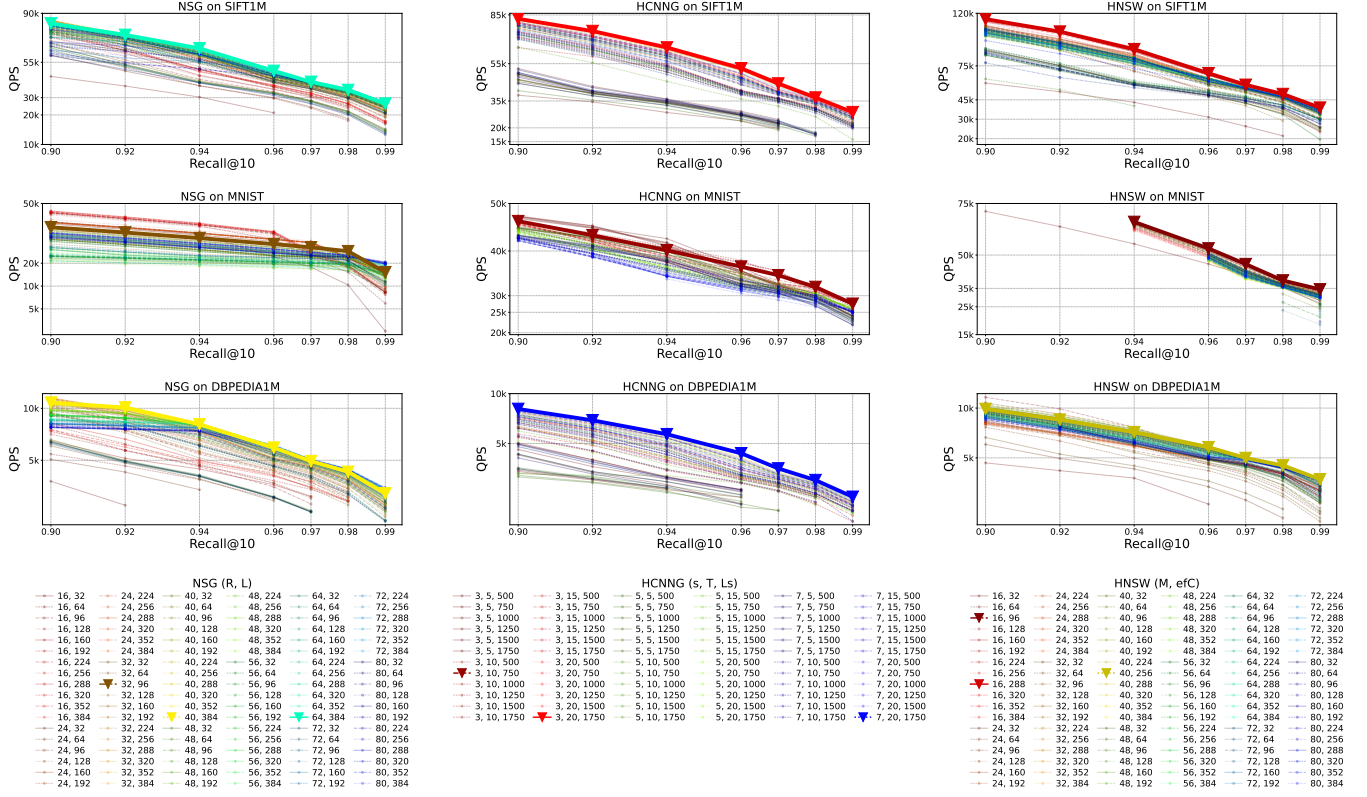


Figure 9: Construction-parameter sweeps of graph indexes on SIFT1M, MNIST, and DBPEDIA1M (QPS–Recall@10). Each curve corresponds to one construction hyperparameter setting of the underlying index (NSG: $R, efConstruction$; HCNNG: s, T, L_s ; HNSW: $M, efConstruction$). To summarize the best tuned reference for each (dataset, index), we select the configuration with the smallest mean QPS rank over the high-recall regime (Recall@10 ≥ 0.9), where ranks are computed at each evaluated recall point (higher QPS is better); the selected configuration is highlighted with enlarged markers for visual emphasis.

K.1 Pruning Ratio: Edit Budget

The pruning ratio ρ is the per-round edit budget for EER-guided pruning. It does not define a new graph topology from scratch; it controls how aggressively PIGR applies trace-supported removals on the current graph. A large ρ removes more low-yield edges at once but can induce coupled rerouting effects and distort later traces, whereas an overly small ρ leaves many inefficient edges untouched and slows down refinement. Figure 10 shows that a small value, around $\rho = 0.02$, is sufficient and stable in our sweeps. We therefore use $\rho = 0.02$ as a conservative default rather than tuning it separately for each dataset-index pair.

K.2 Returned-Nighbor Size: Retrieval Scope

The returned-neighbor size k_{core} controls the locality of the bridging candidates. It should be interpreted together with the target retrieval regime rather than as a free topology-search parameter. In our main experiments, the target metric is Recall@10, so we set $k_{core} = 10$ to focus bridging on the most query-relevant local neighborhood. Larger values expose broader candidate regions, but they also introduce weaker or less relevant neighbors and may add edges that do not help the deployed greedy traversal. Figure 11 supports this localized default. For larger target- k deployments, k_{core}

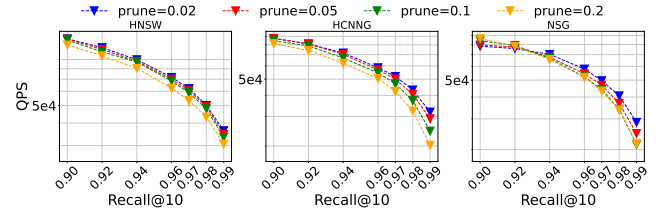


Figure 10: Impact of pruning ratio ρ on PIGR performance (QPS–Recall@10).

can be increased accordingly; otherwise, using an unnecessarily large scope may trade local high-recall efficiency for broader but less targeted connectivity.

K.3 Returned-Nighbor Size: Retrieval Scope

The returned-neighbor size k_{core} controls the locality of the bridging candidates. It should be interpreted together with the target retrieval regime rather than as a free topology-search parameter. In our main experiments, the target metric is Recall@10, so we set $k_{core} = 10$ to focus bridging on the most query-relevant local neighborhood. Larger values expose broader candidate regions, but

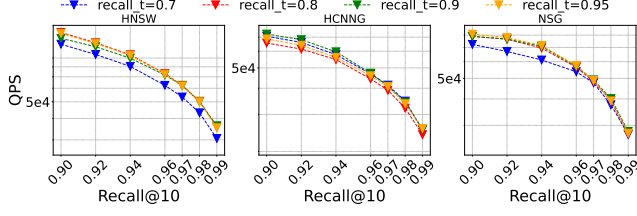


Figure 11: Impact of $recall_{target}$ on PIGR performance (QPS–Recall@10).

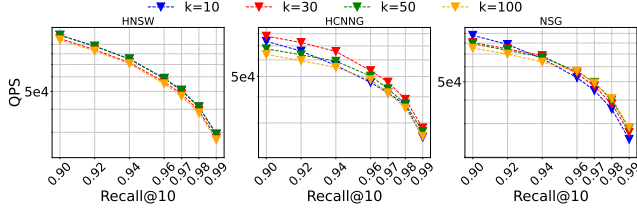


Figure 12: Impact of k size on PIGR performance (QPS–Recall@10).

they also introduce weaker or less relevant neighbors and may add edges that do not help the deployed greedy traversal. Figure 12 supports this localized default. For larger target- k deployments, k_{core} can be increased accordingly; otherwise, using an unnecessarily large scope may trade local high-recall efficiency for broader but less targeted connectivity.

These ablations point to a unified principle: PIGR benefits most from *small, evidence-faithful, and local* edits rather than aggressive global modifications. A conservative pruning ratio stabilizes the feedback loop, a sufficiently high $recall_{target}$ ensures bridging is guided by reliable traces, and a local k scope focuses additions on the most influential neighborhood region.

L An Illustrative Decomposition of Pruning vs. Bridging

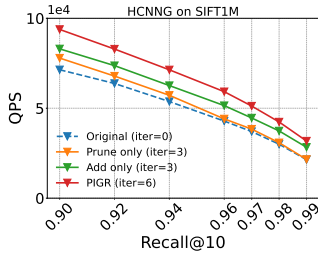


Figure 13: Independent effects of pruning and bridging on HCNG with SIFT1M (QPS–Recall@10) under a matched refinement budget.

This section provides an illustrative example that separates the effects of pruning and bridging under a matched refinement budget. On HCNG with SIFT1M, we compare four variants: the original index, pruning-only refinement, bridging-only refinement, and the full PIGR loop. While this experiment is not meant to establish

broad generality (which is supported by the multi-dataset results in Section 5.2), it serves as a sanity check that both components contribute and that their combination yields the strongest curve.

As shown in Figure 13, pruning-only provides a modest but consistent improvement, reflecting the removal of persistently low-yield edges. Bridging-only yields a larger upward shift, suggesting that targeted shortcuts can more directly reduce backtracking-induced redundancy in this configuration. Enabling both components achieves the best trade-off, consistent with PIGR’s design that uses trace feedback to keep helpful additions while pruning ineffective edges.